

CYBERSECURITY PROJECT REPORT

Advanced Password Security Analyzer

Hashing • Salting • Argon2id • Entropy Analysis • Controlled Dictionary-Attack Demonstration

Project	Advanced Password Security Analyzer
Author	Tanzila Sayyed
Environment	Kali Linux / Python 3
Testing	Pytest + John the Ripper
Repository	GitHub: Password-Security-Analyzer
Status	Core implementation completed

Ethical-use notice: All password analysis and cracking demonstrations described in this report are intended for self-generated test data and authorized laboratory environments only.

1. Executive Summary

The Advanced Password Security Analyzer is a Python-based cybersecurity application developed to demonstrate practical password-security concepts. The system evaluates password complexity, detects common passwords, estimates entropy, generates cryptographic hashes, applies random salting, performs Argon2id password hashing and verification, creates JSON security reports, and demonstrates a controlled dictionary attack using John the Ripper.

The project intentionally combines defensive controls with a restricted offensive-security demonstration. The cracking exercise uses only a self-generated SHA-256 hash and a small test wordlist. No real credentials or third-party systems are required.

2. Problem Statement

Weak, predictable, or improperly stored passwords remain a significant security risk. A practical learning system is therefore required to show how password strength can be measured, why salting matters, why fast hashes such as SHA-256 are unsuitable as a standalone production password-storage mechanism, and how modern password hashing such as Argon2id improves security.

3. Objectives

- Analyze password complexity using multiple security characteristics.
- Identify common and easily guessable passwords.
- Estimate password entropy and classify crack resistance.
- Demonstrate SHA-256 hashing for educational comparison.
- Generate cryptographically secure random salts.
- Implement Argon2id password hashing and verification.
- Demonstrate a controlled dictionary attack against a self-generated hash.
- Automate functional checks using Pytest.
- Generate structured JSON security reports.
- Maintain and publish the project using Git and GitHub.

4. System Architecture

The application follows a modular command-line architecture. The main controller invokes specialized modules for password analysis, entropy, hashing, salting, Argon2id processing, and reporting.

Module	Responsibility
main.py	Unified CLI menu and workflow controller
password_analyzer.py	Complexity checks and password-strength scoring
entropy_analyzer.py	Entropy estimation and resistance classification
hash_engine.py	SHA-256 / MD5 educational hash generation
salt_engine.py	Secure random salt and salted SHA-256 generation
argon2_engine.py	Argon2id hashing and password verification
report_generator.py	JSON security-report generation

Module	Responsibility
tests/	Automated Pytest test cases
results/	Test hashes, wordlist, and generated report

5. Technologies Used

Technology	Use
Python 3	Core application development
hashlib	Cryptographic hash demonstrations
secrets	Cryptographically secure salt generation
argon2-cffi	Argon2id password hashing
Regular Expressions	Password character-category checks
JSON	Structured report output
Pytest	Automated testing
John the Ripper	Controlled dictionary attack
Git / GitHub	Version control and project publication
Kali Linux	Cybersecurity development/testing environment

6. Implementation Details

6.1 Password Strength Analysis

The analyzer checks length, uppercase characters, lowercase characters, digits, special characters, and membership in a small common-password set. A security score is calculated and mapped to a strength category.

Example test result: **T3st!Secure#2026** achieved a score of **6/6** and was classified as **STRONG**.

6.2 Entropy Analysis

The entropy module estimates the theoretical uncertainty of a password using its length and detected character categories. The result is expressed in bits and mapped to a crack-resistance category.

For the test password **T3st!Secure#2026**, the application reported **104.87 bits** of estimated entropy and **VERY HIGH** resistance.

6.3 SHA-256 Hashing

SHA-256 is implemented using Python's hashlib library. It is used in this project as an educational cryptographic-hashing demonstration and as the input format for the controlled cracking exercise.

Security consideration: SHA-256 is intentionally fast. Fast hashes are not appropriate as a standalone mechanism for storing user passwords because attackers can test large numbers of guesses quickly.

6.4 Salting

The salt module uses Python's secrets module to generate a random 16-byte salt represented as hexadecimal text. The password and salt are combined before producing a salted SHA-256 value. This demonstrates how unique salts prevent identical passwords from producing identical salted hashes.

6.5 Argon2id

Argon2id was added as the project's modern password-hashing component. The implementation uses the PasswordHasher interface from argon2-cffi and supports both hashing and verification. The generated format includes the algorithm version, memory cost, time cost, parallelism, salt, and derived hash.

The implemented verification test successfully produced: **Correct password: VERIFIED** and **Wrong password: REJECTED**.

7. Controlled Dictionary-Attack Demonstration

A controlled laboratory test was created using a self-generated SHA-256 hash and a small wordlist. John the Ripper was used to demonstrate the concept of dictionary-based password recovery. The activity was restricted to project-generated test data.

Test wordlist:

```
hello123
password
admin123
welcome
Test@123
qwerty123
letmein
```

The purpose of this exercise is to demonstrate why predictable passwords can be recovered from weakly protected hashes and why modern, slow password hashing algorithms are important.

8. Testing and Validation

Automated tests were created with Pytest to validate core password-analysis behavior. The final test run completed successfully with **5 passed**.

Test	Expected behavior	Result
Common password	password123 → VERY WEAK	PASS
Weak password	hello → WEAK	PASS
Strong password	T3st!Secure#2026 → STRONG	PASS
Uppercase detection	Uppercase check detected	PASS
Special character	Special-character check detected	PASS

9. Sample Security Results

Metric	Observed result
Test password	T3st!Secure#2026 (test data only)
Strength	STRONG
Security score	6/6
Estimated entropy	104.87 bits
Crack resistance	VERY HIGH
Argon2id verification	Correct password VERIFIED
Wrong-password test	Correctly REJECTED
Automated tests	5 passed

10. Project Directory Structure

```
Password-Security-Analyzer/  
├── src/  
│   ├── main.py  
│   ├── hash_engine.py  
│   ├── salt_engine.py  
│   ├── password_analyzer.py  
│   ├── entropy_analyzer.py  
│   ├── argon2_engine.py  
│   └── report_generator.py  
├── tests/  
├── results/  
├── screenshots/  
├── documentation/  
├── README.md  
├── requirements.txt  
└── .gitignore
```

11. Ethical and Security Considerations

The password-cracking component is a controlled educational demonstration. Only self-generated hashes, test passwords, and a local test wordlist should be used. The project must not be used to access or recover credentials from accounts or systems without explicit authorization.

The report and application avoid storing the plaintext test password in the generated JSON security report. For production systems, password storage should use a dedicated password hashing algorithm such as Argon2id, bcrypt, scrypt, or PBKDF2 with appropriate parameters and secure operational controls.

12. Limitations

- Entropy is an estimate and does not model every real-world password-guessing strategy.
- The common-password list is intentionally small and educational.
- The dictionary attack is limited to a small local test wordlist.
- SHA-256 is included for comparison, not recommended as standalone password storage.
- The current application is CLI-based rather than a web dashboard.
- No external breach database is integrated in the current implementation.

13. Future Enhancements

- Configurable Argon2id parameters and security-policy profiles.
- Password breach/reuse detection using a privacy-preserving approach.
- CSV and PDF report export directly from the application.
- Interactive terminal dashboard with visual risk indicators.
- Expanded password-policy and enterprise audit reporting.
- Additional unit and integration tests.
- Containerized deployment for reproducible laboratory environments.

14. Conclusion

The Advanced Password Security Analyzer successfully demonstrates a complete learning workflow around password security: measuring password strength, estimating entropy, comparing hashing approaches, applying salts, using Argon2id for modern password hashing, validating credentials, generating structured reports, and performing a controlled dictionary-attack exercise. The successful automated test suite and GitHub publication provide evidence of a working, modular cybersecurity project.

15. Project Repository

GitHub repository: github.com/tanzilasayyed27-byte/Password-Security-Analyzer

Prepared by: Tanzila Sayyed

Project: Advanced Password Security Analyzer

Report type: Cybersecurity Project Documentation